Tabla de contenidos

1. Levantando excepciones: `raise`
2. Manejo de Excepciones: `try` y `except`
    a. Múltiples excepciones con `except`
    b. Flujos complementarios: `else` y `finally`
    c. Observaciones

Levantando excepciones: `raise`

Hemos visto algunos tipos de excepciones existentes y las condiciones bajo las cuales se gatillan.

Sin embargo, también podemos generar una excepción en el momento que queramos creando una nueva instancia de la excepción, y utilizando la sentencia **`raise`**. Cada excepción tiene un tipo definido, y es posible definir un mensaje explicativo para el usuario.

Por ejemplo, la siguiente función `convert_coordinate` recibe un *string* que sigue el formato `"coord_x,coord_y"` y tiene por objetivo retornar una tupla con los valores enteros de la coordenada. Se podría lanzar excepciones de esta manera:

```
def convert_coordinate(coordinate_string):
    # Si el input no es del tipo esperado
    if not isinstance(coordinate_string, str):
        # aquí se genera una excepción y se incluye información para
el usuario.
        raise TypeError("Coordinate must be a string")

    # Si el input no sigue el formato esperado
    if "," not in coordinate_string:
        # aquí se genera otra excepción y se incluye información para
el usuario.
        raise ValueError("Coordinate must be separated by a comma")

    coord_x, coord_y = coordinate_string.split(",")
    return (int(coord_x), int(coord_y))

text_1 = "56,1123"
x, y = convert_coordinate(text_1)
print(x, y)
```

```
56 1123
```

```
not_text = [43, 3]
x, y = convert_coordinate(not_text)
print(x, y)
```

```
---------------------------------------------------------------------
-----
TypeError                                 Traceback (most recent call
last)
Cell In[4], line 2
      1 not_text = [43, 3]
----> 2 x, y = convert_coordinate(not_text)
      3 print(x, y)

Cell In[2], line 5, in convert_coordinate(coordinate_string)
      1 def convert_coordinate(coordinate_string):
      2     # Si el input no es del tipo esperado
      3     if not isinstance(coordinate_string, str):
      4         # aquí se genera una excepción y se incluye
información para el usuario.
----> 5         raise TypeError("Coordinate must be a string")
      7     # Si el input no sigue el formato esperado
      8     if "," not in coordinate_string:
      9         # aquí se genera otra excepción y se incluye
información para el usuario.

TypeError: Coordinate must be a string
```

```
text_incorrect = "23 - 54"
x, y = convert_coordinate(text_incorrect)
print(x, y)
```

```
---------------------------------------------------------------------
-----
ValueError                                Traceback (most recent call
last)
Cell In[5], line 2
      1 text_incorrect = "23 - 54"
----> 2 x, y = convert_coordinate(text_incorrect)
      3 print(x, y)

Cell In[2], line 10, in convert_coordinate(coordinate_string)
      7 # Si el input no sigue el formato esperado
      8 if "," not in coordinate_string:
      9     # aquí se genera otra excepción y se incluye información
para el usuario.
---> 10     raise ValueError("Coordinate must be separated by a
comma")
     12 coord_x, coord_y = coordinate_string.split(",")
     13 return (int(coord_x), int(coord_y))

ValueError: Coordinate must be separated by a comma
```

Las excepciones interrumpen todo el flujo del programa. Aún si la excepción ocurre dentro de una llamada a una función que ha sido llamada dentro de otras llamadas a función, todas las

llamadas a funciones previas son interrumpidas y la excepción llega hasta el sistema operativo (a menos que sea capturada previamente como veremos más adelante). Notar que en los últimos dos ejemplos no se ejecutó el `print`, debido a que se interrumpió el programa antes.

A continuación, puedes ver otro ejemplo, donde al instanciar un objeto podemos exigir que el inicializador tenga que recibir una tupla de dos elementos como parámetro de entrada, lanzando una excepción cuando recibimos un argumento de otro tipo.

```
class Circle:

    def __init__(self, center):
        if not isinstance(center, tuple):
            raise TypeError("The 'center' parameter must be a tuple")
            # Recordar que cuando ocurre el raise, la ejecución se
interrumpe.
            print("This line is never printed")
        self.center = center

    def __repr__(self):
        return f"El centro es {self.center}"

# Caso correcto donde se crea un objeto usando una tupla.
c1 = Circle((2,3))
c1
```

```
El centro es (2, 3)
```

```
# Caso que genera la excepción al usar una lista como entrada.
c2 = Circle([2,3])
c2
```

```
---------------------------------------------------------------------
-----
TypeError                                 Traceback (most recent call
last)
Cell In[8], line 2
      1 # Caso que genera la excepción al usar una lista como entrada.
----> 2 c2 = Circle([2,3])
      3 c2

Cell In[6], line 5, in Circle.__init__(self, center)
      3 def __init__(self, center):
      4     if not isinstance(center, tuple):
----> 5         raise TypeError("The 'center' parameter must be a
tuple")
      6         # Recordar que cuando ocurre el raise, la ejecución se
interrumpe.
      7         print("Esta línea nunca se imprime")

TypeError: The 'center' parameter must be a tuple
```

Manejo de Excepciones: `try` y `except`

Cada vez que se **levanta** una excepción durante la ejecución del código, es posible **atraparla** mediante el uso de las sentencias `try` y `except`.

La sentencia `try` permite definir un *scope* (bloque de código). Si se levanta una excepción dentro del *scope* de `try`, entonces la excepción es **capturada**. A continuación del bloque de `try` debe haber una o más instrucciones `except`. Las instrucciones `except` permiten implementar el manejo de la excepción capturada.

En el momento que se captura una excepción dentro de `try` el flujo del programa salta inmediatamente al bloque de una de las sentencias `except`. Una vez que el bloque `except` ha terminado, el programa continúa en la instrucción **posterior** al bloque `try`/`except`. El programa **NO regresa** a la sentencia que gatilló la excepción.

Como se mencionó al inicio de esta sección, solo se atrapan excepciones que surgen durante la ejecución del código. Esto implica que excepciones del tipo `SyntaxError` o `IndentationError` no son posibles de **atrapar** porque estas surgen durante la lectura del programa, no su ejecución.

Volvamos al caso de la función `divide`:

```
# No se atrapa ninguna excepción, por lo tanto no se previene un
posible error.
def divide(num, den):
    # Esta función terminará el programa cuando el
    # denominador den sea 0.
    return float(num) / float(den)

r = 4
w = divide(5, r)
print(w)
z = divide(5, r - 4)
print(z)
print("Ya hice ambas divisiones.")
```

```
1.25

---------------------------------------------------------------------
-----
ZeroDivisionError                         Traceback (most recent call
last)
Cell In[15], line 4
      2 w = divide(5, r)
      3 print(w)
----> 4 z = divide(5, r - 4)
      5 print(z)
      6 print("Ya hice ambas divisiones.")

Cell In[14], line 5, in divide(num, den)
```

```
      2 def divide(num, den):
      3     # Esta función terminará el programa cuando el
      4     # denominador den sea 0.
----> 5     return float(num) / float(den)

ZeroDivisionError: float division by zero
```

Primero probaremos "protegiendo" un llamado correcto a `divide`, usando un bloque `try`/`except`.

```
r = 4
try:
    # Dentro de este bloque ejecutamos el código que PODRÍA
    # arrojar una excepción.
    print(divide(5, r))
    print("I already did the division.")

except (ZeroDivisionError) as error:
    # Aquí manejamos la excepción que pueda ser lanzada en
    # el bloque anterior. Si un error del tipo ZeroDivisonError
    # ocurre, se ejecuta este bloque y el resto del programa
    # continúa su ejecución normal. La excepción, como objeto,
    # se puede acceder con la variable error.
    print(f"Error: {error}")
    print("How could you think of dividing by zero? Why are you like
this?")

print("The program continues after the try/except")
```

```
1.25
I already did the division.
The program continues after the try/except
```

En este caso no se generó la excepción, por lo tanto, no se ejecutó el código dentro de `except` y el programa siguió su flujo normal.

Ahora probaremos protegiendo un caso en el que los argumentos de la división genera la excepción, ya que el denominador es 0.

```
# Manejo de la excepción con argumentos inválidos.
# En este caso la función dará un error debido a que el denominador
# utilizado es 0.
r = 4
try:
    print(divide(5, r))
    print(divide(5, r - 4))
    print("I already did both divisions.")

except ZeroDivisionError as error:
```

```
    print(f"Error: {error}")
    print("How could you think of dividing by zero? Why are you like
this?")

print("The program continues after the try/except")
```

```
1.25
Error: float division by zero
How could you think of dividing by zero? Why are you like this?
The program continues after the try/except
```

Podemos observar que cuando se produjo la excepción dentro del llamado `divide(5, r - 4)`, se generó una excepción de tipo `ZeroDivisionError`. Como esta excepción no fue capturada dentro de `divide`, entonces la excepción se propagó al llamado anterior, pero este llamado sí estaba dentro de un `try`. El flujo del programa salta hacia la sentencia `except` que está asociada al tipo de la excepción (`ZeroDivisionError`) y ejecuta lo que hay dentro de ella. Una vez que se ha ejecutado todo lo que había dentro de `except ZeroDivisionError`, el flujo del programa continúa normalmente.

Pongamos atención a dos cosas: (1) la instrucción `print("I already did both divisions.")` nunca se ejecuta porque al capturar la excepción, el flujo salta inmediatamente al `except`; (2) como la excepción fue exitosamente manejada, una vez que el `except` se completó, el programa siguió su flujo normal y, a diferencia de los ejemplos anteriores, no se "cayó".

Cómo se mencionó al inicio de esta sección, solo se atrapan excepciones que surgen durante la ejecución del código. Esto implica que excepciones del tipo `SyntaxError` o `IndentationError` no son posible de **atrapar** en el mismo archivo porque estas surgirán durante la lectura del programa, no su ejecución.

```
# Atrapar SyntaxError en el mismo código --> Fallará.
try:
    print("Anya
except SyntaxError:
    print("SyntaxError detected")
```

```
Anya
```

No obstante, igual existen formas de atrapar estos errores. Una de ellas es utilizar `import` de un posible archivo con errores de sintaxis o indentación, y dejar el `import archivo` dentro de un `try/excepy`. Esta solución logrará atrapar este tipo de error porque durante la ejecución del `import` es que recién se intentará leer el archivo con `SyntaxError`.

En el siguiente ejemplo se dejó el mismo código con error `print("Anya` en un archivo llamado `syntax_error_file.py` y ahora se hará `import` de él, pero atrapando la excepción de `SyntaxError`.

```
# Atrapar SyntaxError desde un import --> Funcionará.
try:
```

```
    import syntax_error_file
except SyntaxError:
    print("SyntaxError detected")
```

```
SyntaxError detected
```

Múltiples excepciones con `except`

Con la sentencia `except` podemos incluir varios tipos de excepciones para manejar. En el siguiente caso, las causas que generarán las excepciones son distintas, pero el tratamiento para todas ellas será el mismo. Si lo que necesitamos es realizar un tratamiento diferenciado según el tipo de excepción, entonces podemos añadir más bloques de `except`, en donde cada uno tiene definido las excepciones sobre las que debe actuar.

```
def convert_coordinate(coordinate_string):
    # Verificamos que el input es del tipo esperado.
    if not isinstance(coordinate_string, str):
        # Intencionalmente levantamos la excepción. Recuerda que las
excepciones son objetos.
        raise TypeError("Coordenada debe ser un string")

    # Verificamos que el input sigue el formato esperado.
    if "," not in coordinate_string:
        # El mensaje incluido en la excepción es el que se despliega
        # cuando la manejamos después.
        raise ValueError("Coordenada debe estar separada por una
coma")

    coord_x, coord_y = coordinate_string.split(",")
    return (int(coord_x), int(coord_y))
```

Ahora manejamos la excepción que pueda ser lanzada durante la ejecución de la función `convertir_coordenada`.

El primer caso levantará una excepción debido a que el argumento no es válido al no ser del tipo esperado (no es de tipo `str`).

```
try:
    print(convert_coordinate([42, 3]))

# En esta parte manejamos las excepciones una vez que son lanzadas.
except TypeError | ValueError as error:
    # Este bloque sólo maneja excepciones del tipo TypeError.
    print(f"Error: {error}")
    print("Check the argument type.")

except ValueError as error:
    # Este bloque sólo maneja excepciones del tipo ValueError.
```

```
    print(f"Error: {error}")
    print("A ValueError occurred. Please verify your values.")

Error: Coordenada debe ser un string
Check the argument type.
```

En este segundo caso, se levantará una excepción porque el argumento no sigue el formato esperado.

```
try:
    print(convert_coordinate("32 - 190"))

# En esta parte manejamos las excepciones una vez que son lanzadas.
except TypeError as error:
    # Este bloque sólo maneja excepciones del tipo TypeError.
    print(f"Error: {error}")
    print("Check the argument type.")

except ValueError as error:
    # Este bloque sólo maneja excepciones del tipo ValueError.
    print(f"Error: {error}")
    print("A ValueError occurred. Please verify your values.")

Error: Coordenada debe estar separada por una coma
A ValueError occurred. Please verify your values.
```

También, es posible capturar varios tipos de errores con un solo bloque `except`. Se logra entregando una tupla de tipos de excepción en vez de solamente un tipo. Como para el siguiente ejemplo de una función divide personalizada, que agrupa los errores `ZeroDivisionError` y `TypeError`.

```
def divide(num, den):
    if not (isinstance(num, int) and isinstance(den, int)):
        raise TypeError("Type error in numerator or denominator. :'(")

    if num < 0 or den < 0:
        raise ValueError("There is a negative value in numerator or
denominator >:(")

    return float(num) / float(den)

for arguments in [(45, "5"), (-23, 5), (4, 0), (21, 7)]:
    try:
        print(f"Calling divide{arguments}")
        print(divide(*arguments))

    # En esta parte manejamos las excepciones una vez que son
lanzadas.
    except (ZeroDivisionError, TypeError) as error:
        # Este bloque maneja excepciones del tipo ZeroDivisionError o
```

```
TypeError.
        print(f"Error: {error}")
        print("Check the division data. There is an incorrect type.\
n")

    except ValueError as error:
        # Este bloque sólo maneja excepciones del tipo ValueError.
        print(f"Error: {error}")
        print("A ValueError occurred. Please verify your values.\n")

Calling divide(45, '5')
Error: Type error in numerator or denominator. :'(
Check the division data. There is an incorrect type.

Calling divide(-23, 5)
Error: There is a negative value in numerator or denominator >:(
A ValueError occurred. Please verify your values.

Calling divide(4, 0)
Error: float division by zero
Check the division data. There is an incorrect type.

Calling divide(21, 7)
3.0
```

Flujos complementarios: `else` y `finally`

El bloque de `try` y `except` puede ser complementado opcionalmente con las sentencias **`else`** y **`finally`**:

- Las instrucciones dentro del bloque `else` se ejecutarán **siempre y cuando no se haya lanzado ninguna excepción**.
- En el bloque de la sentencia `finally` van instrucciones que se realizan **siempre, independientemente de si ocurrió una excepción o no**.

```
# Esta corresponde a la estructura completa de try and except.
try:
    # Probamos si es posible realizar la operación.
    result = divide(10, 0)
    print("This line does not execute if an exception occurs in the
previous line.")

except (ZeroDivisionError, TypeError):
    # Este bloque opera para los tipos de excepciones definidos.
    print("Check the input data. They are either not ints or the
denominator is 0!")

except ValueError:
    # Este bloque sólo maneja excepciones del tipo ValueError.
    print("The input values are negative")
```

```
else:
    # Como no hubo excepciones puede retornar normalmente el resultado
    # En este caso, si se coloca un return después de la operación y
    # esta es correcta, entonces nunca llegará a este punto.
    print("OK! No errors with the data")

finally:
    print("Always remember to use exceptions to handle errors in your
program")
```

```
Check the input data. They are either not ints or the denominator is
0!
Always remember to use exceptions to handle errors in your program
```

En el ejemplo, si no se levanta ninguna excepción, también sería impreso el mensaje en la sección **`else`**. El siguiente ejemplo ilustra eso:

```
# Esta corresponde a la estructura completa de try and except.
try:
    # Probamos si es posible realizar la operación.
    resultado = divide(10, 5)
    print("This line does not execute if an exception occurs in the
previous line.")

except (ZeroDivisionError, TypeError):
    # Este bloque opera para los tipos de excepciones definidos.
    print("Check the input data. They are either not ints or the
denominator is 0!")

except ValueError:
    # Este bloque sólo maneja excepciones del tipo ValueError.
    print("The input values are negative")

else:
    # Como no hubo excepciones puede retornar normalmente el resultado
    # En este caso, si se coloca un return después de la operación y
    # esta es correcta, entonces nunca llegará a este punto.
    print("OK! No errors with the data")

finally:
    print("Always remember to use exceptions to handle errors in your
program")
```

```
This line does not execute if an exception occurs in the previous
line.
OK! No errors with the data
Always remember to use exceptions to handle errors in your program
```

El uso de **`finally`** es común en la ejecución de funciones de limpieza predefinidas, como por ejemplo, el cierre de un archivo después de ser procesado. Si ocurren errores mientras el archivo está abierto, éste quedará abierto. Es importante crear una rutina que asegure que, independientemente de si se lanza o no alguna excepción, el archivo sea cerrado correctamente.

```
from os import path


fid = open(path.join("data", "log.txt"), "w")

try:
    # Probamos si es posible realizar la apertura del archivo.
    # En este caso se debe generar un error por que el denominador
llega a ser 0.
    for i in range(5, -1, -1):
        fid.write(f"{divide(10, i)}")

except (ZeroDivisionError, TypeError):
    # Este bloque opera para los tipos de excepciones definidos.
    print("Error! Check the input data: they are either not ints or
the denominator is 0!")

else:
    print("The file was created successfully!")

finally:
    # Este bloque asegura que el archivo sea cerrado correctamente
    # independientemente de si se produjo el error.
    print("Always remember to close your files")
    fid.close()
```

```
Error! Check the input data: they are either not ints or the
denominator is 0!
Always remember to close your files
```

Una forma equivalente de hacer esto mismo es usar un *context manager* mediante la sentencia **`with`**. El siguiente ejemplo resume esta manera:

```
from os import path


with open(path.join("data", "log.txt"), "w") as fid:
    try:
        # Probamos si es posible realizar la apertura del archivo.
        # En este caso se debe generar un error por que el denominador
llega a ser 0.
        for i in range(5, -1, -1):
            fid.write(f"{divide(10, i)}")

    except (ZeroDivisionError, TypeError):
```

```
        # Este bloque opera para los tipos de excepciones definidos.
        print("Error! Check the input data: they are either not ints
or the denominator is 0!")
```

```
Error! Check the input data: they are either not ints or the
denominator is 0!
```

Observaciones

El manejo de excepciones es otra forma de control del flujo del programa, similar a lo que ocurre con la sentencia `if`. Entonces, ¿qué conviene más usar? ¿`if`/`else` o excepciones?. **Usar excepciones es más recomendable** que usar `if`/`elif`/`else` **para controlar errores**. Siempre es posible crear un sistema de códigos de error manejado por distintas salidas (`return`) de una función o módulo y suficientes combinaciones de `if`/`elif`/`else`. Sin embargo, esto puede generar casos particulares que complejizan, ensucian el diseño, y le quitan flexibilidad a nuestro programa. Además, continuamente hay que estar agregando nuevos códigos de error, lo que dificulta la mantenibilidad de nuestro código.

En general, las principales ventajas de usar excepciones por sobre `if`/`else` son:

- El programador está obligado a darles algún tratamiento, es decir, manejarlas o levantarlas. Mientras que los códigos de error pueden ser ignorados por el programador.
- El código queda más limpio y fácil de leer (recuerden que el código se lee muchas más veces de lo que se escribe).
- Todas las situaciones del programa son manejadas genéricamente, mientras que usando códigos de error tenemos la obligación de crear estructuras de control para cada función que implementemos.
- El manejo de excepciones permite "notificar" a otras aplicaciones sobre este tipo de situaciones, lo que no sería tan simple de lograr usando códigos de error inventados por el programador.
- ¿Por qué importa que el programa no falle inesperadamente?: Muchas veces exponer errores que no se han manejado a usuarios finales puede ser peligroso, ya que se podrían visualizar trozos de código en los outputs de estos.